



GUIA DE IMPLEMENTAÇÃO E TELAS SDK ANDROID AUTTAR

VERSÃO 1.2

Sumário

INTRODUÇÃO	3
1 USO DA DOCUMENTAÇÃO	3
2 EXEMPLO	3
3 SETUP SDK	4
4 LOGS	5
5 INICIALIZAÇÃO	5
6 CONFIGURAÇÕES	5
7 AUTENTICAÇÃO	6
8 CONECTANDO À UI	6
9 GUIA IMPLEMENTAÇÃO PARA TELAS DE CALL-BACK	6
1.1 SHOWQRCODE	6
1.2 HIDEQRCODE	7
1.3 ASKGENERICCAPTURE	7
1.4 ASKFORVALUE	9
1.5 MESSAGEDISPLAY	10
1.6 ASKFORDATE	10
1.7 CLEANDISPLAY	11
1.8 ASKFORCONFIRM	11
1.9 ASKSELECT	12
1.10 SHOWINSERTCARDMESSAGE	13
1.11 SHOWPIN	14
10 INICIANDO TRANSAÇÃO	15
11 RESULTADO TRANSAÇÃO	16
12 SPLIT PAGAMENTOS	17
13 MULTI TRANSAÇÕES	17
14 MULTIEC	18
15 CONFIGURAÇÕES	19
15.1 SUBCOMÉRCIO	19
16 DISPOSITIVOS SUPORTADOS	22
17 CÓDIGOS DE ERROS	22
18 CÓDIGOS ORIGEM DE ERRO	23
19 CÓDIGOS DE ERROS SDK	23

Introdução

A solução SDK Android Auttar herda todas as funcionalidades da solução padrão de TEF, que seguindo as diretrizes de solução completa Auttar, disponibiliza para o ambiente Android uma solução para meio de pagamento Mobile, oferecendo suporte completo a biblioteca EMV, que garante a captura de operações de Crédito e Débito, nas modalidades chip, magnética e aproximação de forma segura, além de pagamento via QR Code e PIX e Tap on Phone.

1 Uso da documentação

Junto a este documento são enviadas três pastas, a pasta “documentation” possui a documentação do SDK em si.

É possível utilizar ela minimamente abrindo o index.html, porém para visualizá-la de forma completa o ideal é executar os seguintes comandos dentro do diretório:

```
npm i
npm run start
```

O mesmo iniciará uma página web, podendo ser acessada em <http://127.0.0.1:8080/> (localhost) Como a vista abaixo:



Nesta página é encontrada a forma de utilizar o SDK em si, que se encontra na pasta “repository”.

2 Exemplo

A pasta “example” possui uma aplicação completa com exemplo de uso do SDK, possuindo transações de crédito e débito, e exemplos de como chamar telas, é um bom ponto de partida para iniciar a implementação.

O exemplo é desenvolvido em Kotlin, possuindo já as permissões necessárias para executar o SDK.

3 Setup SDK

Para utilizar o Auttar SDK é necessário adicioná-lo como dependência no build.gradle da aplicação com base no dispositivo alvo: TapOnPhone, Pinpad ou POS.

```
//Obrigatório
implementation("br.com.auttar.mobile:payment-sdk-debug:$auttarSdkVersion")

//POS
implementation("br.com.auttar.mobile:payment-sdk-bc-pos-debug:$auttarSdkVersion")

//Pinpad
implementation("br.com.auttar.mobile:payment-sdk-bc-pinpad-debug:$auttarSdkVersion")

//TOP
implementation("br.com.auttar.mobile:payment-sdk-top-debug:$auttarSdkVersion")
```

A aplicação integradora deve também adicionar uma solução de desugaring como o exemplo abaixo :

```
android{
    compileOptions {
        coreLibraryDesugaringEnabled true
    }
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.0.4")
}
```

Caso a aplicação seja voltada para POS, é necessário adicionar a tag abaixo ao manifest da aplicação:

```
<activity
    android:name="br.com.avatek.bc.pin.PinFragment"
    android:exported="false"
    tools:node="merge" />
```

4 Logs

Por padrão a escrita de logs em arquivos é desabilitada, porém ela pode ser habilitada utilizando o `IAuttarConfiguration.saveSupportFiles` :

```
val downloadDir = Environment
    .getExternalStoragePublicDirectory(Environment.DIRECTORY_DOWNLOADS)
    .path
Auttar.getConfigSDK().saveSupportFiles(downloadDir) {
    Toast.makeText(
        this,
        "$it\n\nContendo dados para suporte foi salvo em $downloadDir",
        Toast.LENGTH_LONG
    ).show()
}
```

5 Inicialização

Para utilizar o SDK o primeiro passo necessário é inicializa-lo passando uma referencia da sua Android Application. É recomendado que isso seja feito durante o `onCreate()` da sua aplicação garantindo que o SDK seja corretamente configurado antes de outros componentes.

```
class SDKApplication : Application() {

    override fun onCreate() {
        super.onCreate()
        Auttar.initHomolog(this)
    }
}
```

O objeto Auttar possui três métodos de inicialização dependendo do ambiente em que se pretende realizar transações:

- **Auttar.initHomolog()** : Ambiente para testes e homologações.
- **Auttar.initProduction()** : Ambiente para transações reais em ambiente produtivo.
- **Auttar.initStandAlone()** : Ambiente requer configuração manual de um CTF Server.

6 Configurações

O Auttar SDK expõe sua opções de configuração por meio do método `Auttar.getConfigSDK()` que retorna uma instancia da `IAuttarConfigSDK`.

A primeira configuração que deve ser realizada é solicitar permissões essenciais para realização de transações. Recomendamos que isso seja realizado no `onCreate` de sua Activity principal :

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Auttar.getConfigSDK().scanPermissions(this)
}
```

7 Autenticação

A principal interface para interação com o sdk pode ser obtida por meio do método `Auttar.getSDK()` que retorna uma instância da interface `IAuttarSDK`, nela estão disponíveis os métodos de autenticação conforme o exemplo abaixo :

```
Auttar.getSDK().authenticationMonoEC(
    ParameterLoginIntegrationMonoEC(
        user = "username",
        password = "password",
        cnpj = "08697957000140",
        storeCode = null
    ), object : LoginResultListener {
        override fun onResult(result: LoginResultInterface) {
            val isLoginSucessfull = result.code == 0
            val message = result.message
        }
    })
```

8 Conectando à UI

Durante o fluxo transacional o SDK é responsável por iniciar interações como solicitar entrada de dados ou que o usuário realize ações como a aproximação/inserção do cartão. Para isso é necessário que a aplicação integradora implemente e forneça uma instancia da `ClientUserInterface` para o SDK. Essa interface permite que o SDK delegue funções relacionadas à composição da interface para a aplicação integradora enquanto o SDK realiza as chamadas necessárias para completar os fluxos transacionais. Dessa forma é importante garantir que `IAuttarSDK.connectUI` foi corretamente realizado antes do início de uma transação.

```
val clientUserInterface : ClientUserInterface
Auttar.getSDK().connectUI(clientUserInterface)
```

9 Guia implementação para telas de call-back

Tanto no exemplo, quanto na documentação (documentation) vemos a necessidade de implementar a interface **ClientUserInterface**, que nada mais é que a interface para call-backs de telas, a mesma possui métodos que devem chamar diferentes telas na aplicação.

Este guideline traz exemplos de telas para cada uma destas chamadas, sendo exemplos e não necessariamente precisando ser desenvolvidas como cópias fiéis destas telas. Use o seu padrão visual para criar suas próprias telas, utilizando este guideline como inspiração.

1.1 showQrCode

Este método pede para que seja exibida uma tela exibindo um Qr code, tanto para transações de Qr Code como de Pix. Exemplo:



No exemplo a cima, o qr code é enviado via `qrCodeImage`, sendo um bitmap, o valor e o tipo de pagamento são enviados via `transactionValue` e `transactionType` respectivamente e `pinCode` é um código alternativo para digitar (enviado somente em transações de QR code, não é enviado em caso de PIX).

1.2 `hideQrCode`

Este método deve limpar a tela de Qr code e caso precise apagar o arquivo de imagem gerado (opcional).

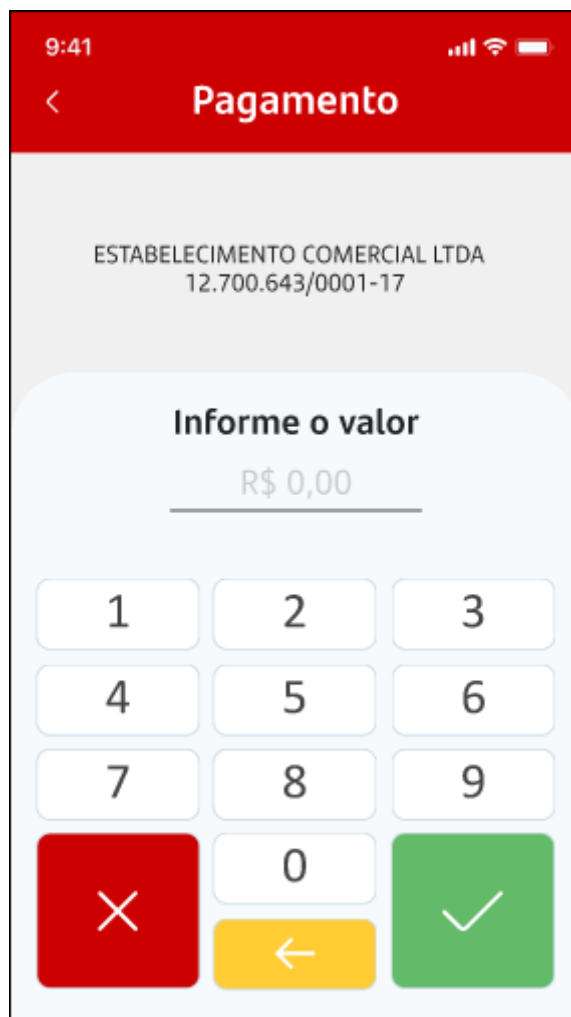
1.3 `askGenericCapture`

Deve abrir uma tela com captura de String genérica, são enviadas mascaras para serem usadas junto, um exemplo é a tela para captura de CPF e telefone, exemplo:

The screenshot shows a mobile application interface. At the top, a red header bar contains a back arrow and the text 'Recarga de Celular'. Below the header, the text 'Sercomtel-Fixo - Getnet' is displayed. A light blue rounded rectangle contains the label 'DDD e Telefone' and the phone number '(12) 98888-7777' which is underlined. At the bottom, there is a numeric keypad with buttons for digits 1-9, 0, a backspace button (X), and a confirmation button (checkmark).

1.4 askForValue

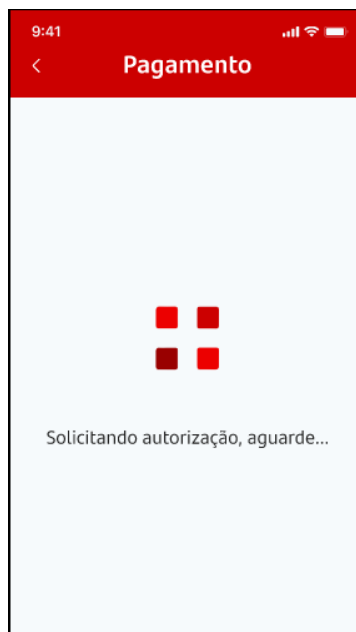
Deve abrir uma tela com captura de valor monetário (R\$), exemplo:



Onde neste caso o parâmetro display da função enviou:
"ESTABELECIMENTO COMERCIAL LTDA 12.700.643/0001-17"

1.5 messageDisplay

Tela que deve exibir uma mensagem em texto vinda pelo parâmetro **message**, como na imagem abaixo:



1.6 askForDate

Tela para captura de datas ou horas (ver o parâmetro **mask** para identificar o tipo), exemplo:



1.7 cleanDisplay



Deve limpar a tela atual, recomendamos deixar uma tela de aguarde nesses momentos, por exemplo:

No exemplo ao lado deixamos uma animação de loading, recomendamos isto para dar uma experiência mais agradável ao usuário, porém isto é opcional.

1.8 askForConfirm

Exibe uma tela de confirmação com a mensagem de display, exemplo:



1.9 askSelect

Deve exibir uma lista, onde o usuário poderá escolher um dos itens, exemplo:



1.10 showInsertCardMessage

Deve exibir uma tela para a leitura do cartão, nesta tela é possível que o display retorne mensagens como: "passe o cartão", "aproxime, insira ou passe o cartão", "insira ou passe o cartão", ou "Aproxima o cartão":



Fique a vontade para incluir animações ou ilustrações para que esta tela fique mais agradável para o usuário.

1.11 showPin

Tela de senha, esta tela deverá o display, enquanto espera a digitação de senha do usuário

Obs.: No uso de POS ou Tap On Phone recomenda-se deixar a mensagem na parte superior da tela, pois o teclado virtual do POS irá abrir ocupando cerca de 40% da tela na parte inferior.

Não é necessário desenvolver o teclado, o mesmo será exibido como o exemplo abaixo pelo SDK, ou será pedido no Pinpad Bluetooth (caso esteja usando um).

Exemplo:



10 Iniciando Transação

O método `IAuttar.paymentTransaction` inicia qualquer tipo de transação com base no `DefaultData<AuttarTypeTransaction>` passado como argumento, o exemplo abaixo é uma transação de crédito

```
Auttar.getSDK().paymentTransaction(
    CreditData(
        BigDecimal(10.00),
        TypeCreditTransaction.CREDITO_GENERICO
    )
)
```

A interface `DefaultData` possui diferentes implementações para cada tipo de transação disponível, sendo `AuttarTypeTransaction` uma sealed interface que representa o código de cada operação TEF.

DefaultData	AuttarTypeTransaction	Descrição
CreditData	TypeCreditTransaction	112 – A Vista 113 – Crédito Parcelado Lojista (sem juros) 114 – Crédito Parcelado Administradora (com juros) 119 – Pré-Autorização 120 – A Vista Digitado 121 – Crédito Digitado Parcelado Lojista (sem juros) 122 – Crédito Digitado Parcelado Administradora (com juros) 132 – Captura Pré-autorização 167 – Captura Pré-autorização Parcelado Lojista (sem juros) 168 – Captura Pré-autorização Parcelado Administradora (com juros) 223 – Crédito genérico 412 – Alteração Pre Autorização
DebitData	TypeDebitTransaction	101 – Débito 103 – Débito pré-datado 224 – Débito genérico 339 – Pagamento de Carnê
VoucherData	TypeVoucherTransaction	106 - Voucher

CancellationData	TypeCancellationTransaction	128 – Cancelamento (Crédito, Débito, QR Code e etc.)
FinalizeData	TypeFinalizeTransaction	6 – Confirmação 7 – Desfazimento
RechargePhoneData	TypeRechargePhoneTransaction	174 – Compra crédito digital 283 – Consulta telefone
QRCodeData	TypeQRCodeTransaction	420 – Pagamento com QR
PixData	TypePixTransaction	422 – Pagamento via PIX 423 – Consulta o status de uma transação PIX 431 – Devolução via Pix
InvoiceData	TypeInvoiceTransaction	348 – Consulta Fatura Cartão Detalhada 183 – Pagamento Fatura Cartão

Operações restritas a funções administrativas devem usar o administrativeTransaction

```
Auttar.getSDK().administrativeTransaction(
    InitializeData(TypeInitializeTransaction.INICIO_DIA)
)
```

DefaultData	AuttarTypeTransaction	Descrição
InitializeData	TypeInitializeTransaction	225 – Início de dia 400 – Carga de tabelas forçada 415 – Teste de comunicação
PixQueryData	TypePixTransaction	422 – Pagamento via PIX 423 – Consulta o status de uma transação PIX
MultiECQueryData	TypeMultiEcTransaction	329 – Consulta estabelecimentos MultiEC

11 Resultado Transação

Depois que uma transação é iniciada, o SDK fica responsável por solicitar os dados necessários dado que a conexão com a UI foi corretamente estabelecida.

O resultado da transação será emitido por meio de um Flow, que pode ser coletado conforme demonstrado no exemplo abaixo :

```
Auttar.getSDK().transactionStateFlow.collect { transactionState:
AuttarTransactionState ->
    if (transactionState is AuttarTransactionState.Done) {
        val tefResult: TefResultInterface = transactionState.result
    }
}
```

12 Split Pagamentos

Transações do tipo Crédito e Débito possuem uma propriedade publicada `splitPaymentIntegration` que pode ser configurada para habilitar a função de split de pagamentos.

```
Auttar.getSDK().paymentTransaction(
    CreditData(
        BigDecimal(0),
        TypeCreditTransaction.CREDITO_GENERICO
    ).apply {
        with(splitPaymentIntegration) {
            setSellerId("SellerID")
            setOrderId("OrderId")
            addProduct(
                code = "productCode",
                amount = 10,
                valorUnitario = BigDecimal(10.00),
                descricao = "ProductDescription",
                idSubSeller = "SubsellerId",
                valorJuros = null,
                valorPorcentagem = BigDecimal(5.00),
                valorTaxa = BigDecimal(1.00)
            )
        }
    }
)
```

13 Multi Transações

Nos cenários que exigem múltiplas operações TEF, como por exemplo um pagamento realizado com mais de um cartão, a aplicação deve solicitar o início de uma multi-transação por meio do método `IAuttarSDK.multiTransactionStarted()`, antes do início das transações. Essa chamada desabilita a confirmação automática das transações subsequentes exigindo que as mesmas sejam manualmente confirmadas por meio do `IAuttarSDK.finalizeTransaction()` para cada operação realizada. O fluxo multi transação é finalizado com a chamada `IAuttarSDK.multiTransactionFinished` que por meio de call-backs disponibiliza os TEF results.

```
Auttar.getSDK().multiTransactionStarted()
...
//As many transactions as necessary
Auttar.getSDK().paymentTransaction()
...
Auttar.getSDK().multiTransactionFinished(object : FinalizeTransactionListener {
    override fun onSuccess(data: Array<FinalizeDataInterface>) {
        if (data.isNotEmpty()) {
            data.forEach { finalizeData ->
                val tefResult = finalizeData.tefResult
                auttarSdk.finalizeTransaction(finalizeData)
            }
        }
    }
})
```

```

    }
}

override fun onError(errorCode: String, message: String, action: String) {
    //Handle errors
}

})

```

14 MultiEC

O SDK pode ser configurado para processar múltiplos estabelecimentos em um mesmo terminal. A primeira alteração necessária para habilitar esse fluxo é a autenticação utilizando `IAuttarSDK.authenticateMultiEC()`.

```

Auttar.getSDK().authenticationMultiEC(ParameterLoginIntegrationMultiEC(
    user = "username",
    password = "password",
    groupEC = "Group EC",
    codeActive = "codeActive"
), object : LoginResultListener {
    override fun onResult(result: LoginResultInterface) {
        val isLoginSucessfull = result.code == 0
        val message = result.message
    }
})

```

Posteriormente é necessário realizar uma consulta para obter a lista dos estabelecimentos disponíveis por meio da transação administrativa abaixo :

```

Auttar.getSDK().administrativeTransaction(
    MultiECQueryData(BigDecimal(0),
TypeMultiECTransaction.CONSULTA_ESTAB_MULTI_EC)
)

```

A lista de estabelecimentos será retornada como resultado da transação e deve ser salva pela aplicação.

```

Auttar.getSDK().transactionStateFlow.collect { transactionState ->
    if (transactionState is AuttarTransactionState.Done) {
        transactionState.result.auttarMerchantMultiEC?.forEach { identifier ->
            val multiECIdentifier = identifier
        }
    }
}

```

Com o identificador multi ec salvo é possível iniciar as transações alterando a propriedade `identifierMultiEC` no `DefaultData` correspondente.

```

auttarSdk.paymentTransaction(
    CreditData(
        BigDecimal(0),
        TypeCreditTransaction.CREDITO_GENERICO
    ).apply { this.identifierMultiEC = multiECIdentifier }
)

```

15 Configurações

Alguns dados de configuração são acessíveis através da interface IUserConfiguration, disponível pela propriedade userConfiguration da fachada IAuttarConfigSDK contida no objeto Auttar.

```
val configSDK: IAuttarConfigSDK = Auttar.getConfigSDK()
```

Os dados disponíveis para consulta estão relacionados ao terminal registrado, como no exemplo abaixo, onde temos uma consulta de CNPJ vinculada à autenticação realizada anteriormente:

```
val configSDK = Auttar.getConfigSDK()
val CNPJ = configSDK.userConfiguration.CNPJ.value
```

15.1 Subcomércio

É possível configurar o SDK para utilizar um **subestabelecimento (submerchant)** nas transações processadas pelo SDK. Os dados do submerchant são definidos através da fachada de configuração mostrada acima, utilizando o método:

```
/**
 * Set the data for sub merchant transactions.
 * @param subMerchant Data class with the sub merchant information.
 * @return Task result with code = 0 in case of success, otherwise
 * code = -1
 */
suspend fun setSubMerchant(subMerchant: SubMerchant) :
SubMerchantResult
```

A classe de dados SubMerchant deve conter todas as informações necessárias, conforme descrito na tabela abaixo:

Parameter	Type	Max. Size	Details
id	String	15	Alphanumeric submerchant ID. Example: 1248ADC
city	String	13	Submerchant city, only letters allowed (a-z and A-Z).

			Example: SAO PAULO
state	String	2	Submerchant state, only letters allowed (a-z and A-Z) Example: SP
zipCode	String	8	Submerchant zip code, numbers only. Example: 90123456
cpfCnpj	String	11/14	Submerchant CPF or CNPJ, numbers only. Example: 12345678901234
address	String	40	Submerchant address, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Example: AV NACOES UNIDAS, 1245
razaoSocial	String	99	Submerchant razao social, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Exemplo: GETNET ADQUIRENCIA E SERVICOS PARA MEIOS DE PAGAMENTO S.A
nomeFantasia	String	99	Submerchant fantasy name, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Example: Loja Santo Antônio da Pat
phone	String	11	Submerchant phone, numbers only. Example : 11987654321
url	String	255	Submerchant URL, mandatory only for e-commerce. Example: https://www.example.com.br OPTIONAL
idTransacao	String	40	TAG for partner internal control. Example: 47144560905_717e756e666d3e777833 OPTIONAL
idPagamento	String	20	TAG for partner internal control. Example: 000087806827933 OPTIONAL

Após definir o submerchant, o objeto SubMerchantResult será retornado com o resultado da operação. Em caso de erro, o campo message conterá uma string descrevendo o problema.

```
data class SubMerchantResult(
    var code: Int,
    var message: String? = null
)

enum class SubMerchantResultCodeEnum(val codeResult: Int) {
```

```
INVALID_FIELD(-1),  
VALID(0)  
}
```

IMPORTANTE: O SDK manterá o último submerchant definido e o utilizará automaticamente em todas as transações executadas. Definir um novo substituirá o anterior.

Você pode verificar as informações do submerchant em uso pelo SDK através da fachada de configuração utilizando o método abaixo:

```
/**  
 * Function to return the registered sub merchant  
 * @return Returns the registered object SubMerchant or null if none  
 is registered.  
 */  
suspend fun getSubMerchant() : SubMerchant?
```

Para remover o submerchant e utilizar o método padrão de transação, você pode usar o seguinte método da mesma fachada:

```
/**  
 * Remove the stored sub merchant data  
 */  
suspend fun clearSubMerchant()
```

16 Dispositivos suportados

- PIN PAD bluetooth:

Terminal	Fabricante
Link 2500	Ingênico
MP5	Gertec
D150	PAX
D200	PAX
ICMP	Ingênico
ME30S	Newland

- POS Android:

Terminal	Fabricante
APOS A8	Ingênico
DX8000	Ingênico
GPOS700	Gertec
N910	Newland
SUNMI	P2
SUNMI	P3

17 Códigos de erros

Os códigos de erros são formatados para que seja possível identificar a origem do erro e se é um erro mapeado. Veja o exemplo abaixo

UC20:5331

	U	C	20	5331
Description	Se error code começa com U ele não foi mapeado pelo SDK e é um erro inesperado.	Este campo representa a origem do erro conforme a tabela de Código de Origem Erro.	Código de erro do retornado pelo CTF, BC ou atestatação.	Código de erro do cliente ou da rede.
Type	Char	Char	Int	String
Obrigatório	False	True	True	False

18 Códigos Origem de Erro

Código Origem Erro			
Char	Origem	Enum	Detalhes
A	Erro Atestação	CodeErrorAttestationEnum	Somente para TapOnPhone
B	BC	CodeErrorBCEnum	Segue tabela de erros Pinpad ABECS
C	CTF	CodeErrorClientEnum	
N	Network	CodeErrorConnectionEnum	
S	SDK	CodeErrorSDKEnum	Erro Gerado pelo SDK conforme Código de erros SDK.

19 Códigos de erros SDK

Código	Nome	Descrição
-1	INPUT_TRANSACTION	Erro de parâmetros de transação
-2	PROCESS_CLIENT	Erro inesperado durante fluxo de pagamento
-3	DB_TRANSACTION_NO_DATA	Tentativa de finalizar uma transação quando não existe registros para o dia
-4	DB_TRANSACTION_NOT_PENDING	Tentativa de finalizar uma transação que não está pendente
-5	DB_TEF_RESULT_NO_DATA	Falha ao encontrar Tef Result para finalização de Multi Transaction
-6	MULTI_TRANSACTION_RUNNING	Multi Transaction em execução
-7	MULTI_TRANSACTION_NO_DATA	Erro ao finalizar Multi Transaction
-8	STANDALONE_NOT_CONFIGURED	Inicialização da SDK feita de forma incorreta
-9	HOMOLOG_PROD_NOT_CONFIGURED	Inicialização da SDK feita de forma incorreta

-10	NFC_COMPATIBILITY_ERROR	Funcionalidade NFC não reconhecida no dispositivo.
-11	NFC_DISABLE	Funcionalidade NFC desabilitada
-12	TOP_DND_UNAUTHORIZED	Permissão Do Not Disturb do Android não concedida
-14	PIN_PAD_BLUETOOTH_DISABLED	Pinpad Bluetooth desativado
-15	TOKEN_EXPIRED_ERROR	App Token Expirado
-16	LOCATION_PERMISSION_DENIED	Permissões de localização não concedidas
-17	LOCATION_DISABLE	Localização desabilitada no dispositivo Android
-18	DB_TEF_RESULT_NSU_NOT_FOUND	Registro da transação não encontrada pelo NSU informado
-19	TIME_OUT_WAIT_KEYS	Chaves não retornaram para o dispositivo durante a transação
-61	KEYSTORE_NOT_VALID	Biblioteca SoftTRSM falhou em criar/utilizar a Keystore localmente para atestação.
-62	DEVICE_NOT_SAFE	Biblioteca SoftTRSM retornou que o Device não é seguro por motivo desconhecido.
-63	UNKNOWN_ATT_ERROR	Falha não mapeada na atestação
502	BAD_GATEWAY	Servidores do A&M retornaram 502 para as operações de Atestação, Info e Chaves
-2024	CLIENT_ERROR_ON_TOP	Uso interno
3000	CANCEL_USER_PASSWORD	Transação cancelada pelo usuário durante a inserção de senha

3001	CANCEL_USER_BACKGROUND	Transação cancelada pelo usuário durante a transação
2000	GENERIC	Erro genérico de atestação
2005	TAMPER_CAMERA	Uso da câmera do dispositivo foi detectada durante a transação
2016	TAMPER_SENSORS_FAILED	Falha no uso de sensores para geração de chaves aleatórias para transação
2004	TAMPER_SETTINGS	Configurações sensíveis do Android detectadas durante a transação
0010	INTERNET_ERROR	Falha na comunicação com os servidores do A&M devido à problema de rede